

Adaptive Multi-Agent Orchestration with Signature-Based Selective Re-execution for LLM Workflows

J A L Dhiraj
UG Student

Computer Science and Engineering
Velammal College of Engineering and
Technology.
Madurai, India.
dhirajeng53@gmail.com

N Nithersan
UG Student

Computer Science and Engineering
Velammal College of Engineering and
Technology.
Madurai, India.
nithersan3@gmail.com

S Sahebzathi
Assistant Professor

Computer Science and Engineering
Velammal College of Engineering and
Technology.
Sahebzathi@velammalcollege.edu.in

Abstract — Most multi-agent frameworks built on large language models run a task as a fixed pipeline. If the requirement changes midway, the usual response is to restart the whole workflow, even the parts the change never touched. That wastes time and tokens, and it happens on every framework we looked at. Existing systems recover when a step fails, but none of them ask a simpler question: if nothing failed and the requirement just changed, what actually still holds? This paper describes an orchestrator that answers that question directly. It models a task as a directed acyclic graph and re-executes only the part of it a change actually invalidates. Each step is given a signature by hashing its own definition together with the exact outputs of whatever it depends on; a step only re-runs if it was forced, has no cached result, or its signature has changed. Because the hash includes outputs rather than just structure, a change propagates on its own and stops as soon as a re-run produces the same result as before. The orchestrator also routes tools by capability, falling back to an always-available local tool when a remote one fails, and it pauses irreversible actions for human approval without spending extra tokens to do so. Across seven benchmark scenarios, run against a restart-everything baseline and an equivalent LangGraph implementation, this cut step executions by 24% (54 versus 71) and token use by 22% (19,196 versus 24,502), rising to 40% for late or no-op changes. An ablation that swaps the output signature for ordinary dependency-cone invalidation shows where that saving actually comes from: hashing what a step produced, not just which steps sit downstream of it. The prototype passes 610+ automated tests at roughly 90% coverage and talks to live Model Context Protocol servers rather than mocks.

Keywords— *Multi-Agent Systems, Large Language Models, Workflow Orchestration, Directed Acyclic Graph, Selective Re-execution, Incremental Computation, Tool Fallback, Model Context Protocol.*

I. INTRODUCTION

Large language models are no longer used only as single assistants. Increasingly they are wired together into teams of agents that plan a task, write the code, call whatever external service is needed, and check each other's work. AutoGen [2], MetaGPT [3] and LangGraph [10] are the frameworks most people reach for, and they already show up in software construction, data analysis and enterprise automation pipelines. Otoum and Elkhaili [8] survey this space and note how quickly it has converged on essentially one execution model.

That model assumes the workflow is fixed once planning is done: decompose the task into ordered stages, run them, and treat a step that throws an error as the only thing worth reacting to. In

an actual project this assumption breaks constantly. Someone asks for PostgreSQL instead of MongoDB after the build is already underway, and nothing has technically failed. Every stage ran fine. It is just that some of what they produced is now wrong, and some of it is still perfectly usable, and the framework has no way to tell which is which.

What happens instead is a restart. The whole pipeline runs again, including the stages the change never came near. Every one of those unnecessary re-runs is a fresh model call, so the cost is not just time but tokens, and it grows with the size of the workflow. We saw this directly in our own measurements: a database change midway through a five-step build made both baseline systems in Section V run all ten steps again, when six would have done.

The real gap here is not architectural, it is analytical. Nobody has built a mechanism that can look at a step's inputs and its cached output and decide whether the two still agree. The problem resembles incremental computation in build systems, but with one complication that build systems do not have: an agent's output is free-form text, so there is no way to predict from the inputs alone whether two runs produced the same thing. You have to actually check.

This paper makes the following contributions.

- **Signature-based change propagation** — a step's hash covers its own definition plus its dependencies' exact outputs, so invalidation spreads on its own and stops the moment a re-run gives back an identical result. Section V-D's ablation shows this is something plain dependency analysis just can't do.
- **Capability-equivalent tool fallback** — tools are grouped by capability into chains that end in an always-available local tool, so a workflow can finish even with every remote service down.
- **Planner-generated specialist agents** — roles get made up for each task rather than picked off a fixed list.
- **Pre-execution requirements analysis** — the system reports what credentials, servers and binaries it needs, and decides whether it can actually run, before a single token is spent.
- **Zero-cost approval of irreversible actions** — the payload stays put across the pause, so saying yes resumes execution rather than asking the model to generate it all over again.

Section II goes through existing multi-agent and tool-using systems and points out where the gap is. Section III lays out the system architecture. Section IV is the method itself — the signature, the re-execution rule, and a worked example. Section V is where the numbers are: seven benchmark scenarios and an ablation study. Sections VI and VII cover what we would do next and wrap up.

II. LITERATURE SURVEY

A. Multi-Agent LLM Frameworks

AutoGen [2] treats collaboration as conversation: configurable agents exchange messages until the task is judged done. It is a general idea and it has caught on widely, but the coordination lives in the dialogue itself rather than in any explicit dependency structure. That has two costs. Token use tracks how long the conversation runs rather than how much work actually got done, and there is simply no structure to hang a reuse decision on.

MetaGPT [3] borrows standard operating procedures from software engineering and puts agents on an assembly line with fixed roles — product manager, architect, engineer — and defined handoffs between them. It measurably improves code quality on the usual benchmarks. But the pipeline is fixed at design time. A requirement change sends the task back to the top of the line, and the only roles on offer are the ones the framework's authors thought to build in.

HuggingGPT [7] takes a different approach: a language model acts as controller, picking expert models out of a public repository and sequencing their calls. The planning part really is dynamic, and tool choice is open. What it does not do is keep the plan around afterwards, so a related or repeated request starts from nothing every time.

B. Single-Agent Reasoning and Tool Use

ReAct [4] interleaves reasoning with tool calls so a single agent can ground its reasoning in what it just retrieved, which helps with factuality. But it is one agent following one line of reasoning. There is no orchestration across steps and nothing to fall back on when an action fails.

Reflexion [5] adds a form of self-reflection — a failed attempt gets described in words and retried with a better strategy, which does raise success rates on tasks that allow multiple tries. But the retry stays local to the step that failed. It can tell you why that one step went wrong; it cannot tell you what else, downstream, is now suspect because of it.

Toolformer [6] teaches a model when to call an API through self-supervision, which is neat because it removes hand-written tool-use prompts. The binding between a task and its tool is still fixed once inference starts, though. If the tool goes down, the call fails, full stop — there is no equivalent tool waiting to take over.

C. Workflow-Oriented Orchestration

The closest prior work is Deng et al. [1], our base paper, which models multi-agent AutoML execution as a directed acyclic graph. A workflow orchestrator parses the requirement, builds or retrieves the task graph, and hands subtasks to specialised agents at runtime. Each subagent checks that its inputs are available, that its capability actually matches the task, and that constraints are met, before it executes, and reports back with structured diagnostics if something goes wrong. The orchestrator then scores the whole workflow and repairs it through targeted replanning rather than starting over. We build on the graph model and the capability-matching idea directly; what we add is a way to decide what can be reused.

Otout and Elkhaili [8] survey agentic software engineering more broadly and find the same pattern holding across the field: almost all the robustness work is about detecting and recovering from failure. What happens to a workflow when the specification changes and nothing has actually failed barely comes up.

D. Comparative Summary

TABLE I: Capability Comparison of Related Systems

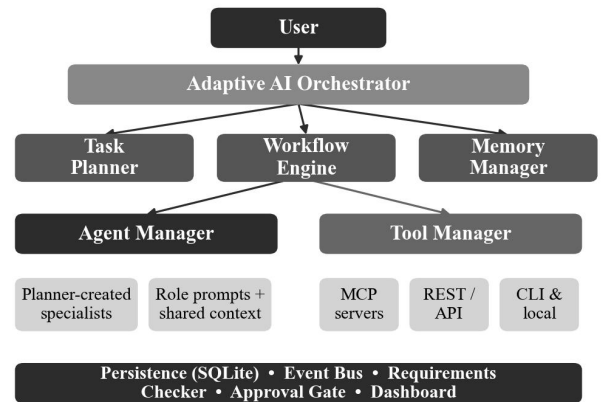
System	Multi-agent	Explicit graph	Reuse on change	Tool fallback
AutoGen [2]	Yes	No	No	No
MetaGPT [3]	Yes	Fixed SOP	No	No
ReAct [4]	No	No	No	No
Reflexion [5]	No	No	No	No
Toolformer [6]	No	No	No	No
HuggingGPT [7]	Yes	Per-run plan	No	No
Deng et al. [1]	Yes	Yes (DAG)	No	Verify only
Proposed	Yes	Yes (DAG)	Yes	Yes

E. Research Gap

Table I lays this out plainly. Every system's idea of adaptation starts with a step failing: something errors, and then the system diagnoses it, retries it, or replans around it. None of them handle the case where nothing fails and the requirement changes anyway, which leaves the workflow holding a mix of stale and still-good results with no way to tell them apart — so adaptation just falls back to restarting, and whatever was still valid gets thrown away along with everything else. The same fixed-pipeline assumption produces three more problems: tool bindings don't move, so one unavailable tool stops the whole workflow; nothing gets reused between related runs; and actions that can't be undone — sending an email, applying a migration — go ahead without anyone signing off on them. This paper's algorithm is built to close all of these at once, not one at a time.

III. SYSTEM ARCHITECTURE

We split the orchestrator into three layers. A control layer takes the task in and owns the workflow from start to finish. Below that, a coordination layer — the Task Planner, Workflow Engine and Memory Manager — decides what runs and in what order. An execution layer, the Agent Manager and Tool Manager, actually does the work. A set of services sits underneath all three and cuts across them: persistence, an event bus, a requirements checker, the approval gate, and the dashboard. Fig. 1 shows how these fit together.



*Agents and tools are created and selected on demand;
no role or tool binding is fixed at design time.*

Fig. 1. Layered architecture of the proposed orchestrator.

A. Task Planner

The planner takes a plain-English task and turns it into a set of steps, each one carrying a description, an assigned role, a required

capability, and a list of what it depends on. It also decides what specialist roles the task needs on the fly — a clinical task might get a trial-statistician role, a legal one an employment-lawyer role, and neither requires touching the framework. Before anything runs, the plan is checked as a directed acyclic graph; if it has a cycle or a dependency pointing at nothing, the whole plan is rejected rather than run partway.

B. Agent Manager

One agent gets created per role the planner declared, each with its own system prompt and access to whatever upstream outputs it needs. Agents come into existence when the workflow needs them and go away once it finishes, so at any moment the active set matches the task at hand rather than some fixed roster decided in advance.

C. Tool Manager

Steps don't name a tool directly — they declare a capability, and the Tool Manager decides which provider actually handles it. For each capability it keeps an ordered chain of providers and picks the first one that's alive. Those providers can be Model Context Protocol [9] servers reached over stdio, SSE or streamable HTTP, authenticated REST endpoints (bearer, basic, API-key, or OAuth2 client-credentials), or plain local command-line tools.

D. Memory, Persistence and Observability

Every result a step produces gets written to an embedded SQLite database along with the signature it was produced under. That record is what makes reuse possible across separate runs, and it's also what lets an interrupted workflow pick back up where it left off. An event bus publishes every state change as it happens, and a web dashboard listens over WebSocket and draws the dependency graph live, marking each step as it gets reused or re-run.

IV. METHODOLOGY

A. Dependency Graph and Execution Levels

Steps get topologically ordered, and any steps that share no ancestor are grouped into the same execution level and run in parallel across a worker pool — independent branches shouldn't have to wait on each other. This level assignment only needs to happen once per plan, since it depends purely on graph shape, not on whether anything actually re-runs later.

B. Input Signatures

Take a step s . Write $D(s)$ for its definition — description, role, required capability, declared inputs — and $dep(s) = (d1, \dots, dk)$ for the steps it depends on, in order. $O(d)$ is whatever output dependency d produced. We define the signature of s as equation (1):

$$\text{Sig}(s) = H(D(s) || O(d1) || O(d2) || \dots || O(dk)) \quad (1)$$

H is a cryptographic hash and $||$ is ordered concatenation. The point of building it this way is that $\text{Sig}(s)$ depends on what the dependencies actually produced, not just on which steps they are. So it's not recording "depends on step X" — it's recording exactly what step X's output looked like the last time this was cached.

C. Selective Re-execution

A step runs only when equation (2) is true:

$$\text{run}(s) \Leftrightarrow \text{forced}(s) \text{ OR } \text{cached}(s) = \text{NULL} \text{ OR } \text{Sig}(s) \neq \text{Sig}_{\text{prev}}(s) \quad (2)$$

This gets us two things we actually care about. Change propagates on its own: edit one step, its output changes, that changes the signature of everything depending on it, and those re-run too, cascading down the graph without anyone having written a rule that says "changing X affects Y." Nobody has to keep an impact map up to date, because there isn't one to keep up to date.

The other half is that the cascade isn't guaranteed to run to completion. If a re-executed step happens to produce exactly the same output as before, everything downstream of it sees an unchanged signature and stops right there. A change only ever touches the part of the graph it genuinely affects, which can be a lot smaller than its full dependency cone. Fig. 2 shows this on a simple chain: editing A pushes through B and C, but C happens to reproduce its old output, so D's signature never moves and the cascade just stops before it. This isn't a minor detail — Section V-D measures exactly how much it matters.

A changes; C re-runs but reproduces its previous output

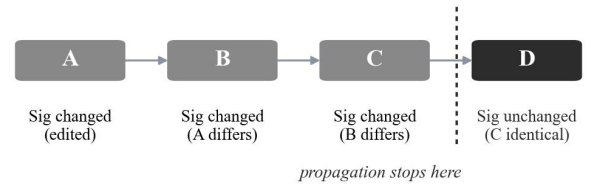


Fig. 2. Signature propagation terminating at an unchanged output.

D. Worked Example

Take a five-step build: requirements analysis, frontend, backend, database schema, and integration testing, where testing depends on the three implementation steps and all of those depend on requirements. Say the client asks for PostgreSQL instead of MongoDB after a full run has already completed.

The database step's definition changes, so its signature changes and it re-runs. Its output is now different, which changes testing's signature too, so testing re-runs as well. Frontend and backend only depend on requirements, and requirements never changed, so their signatures still match and both come straight from cache — no model call needed. Out of five steps, two actually run and three don't have to. If the requirements step itself had changed instead, every signature downstream of it would change too, and all five steps would run — which is the right answer, since all five would genuinely be stale by then. Fig. 3 shows how the graph splits.

Requirement change: MongoDB → PostgreSQL

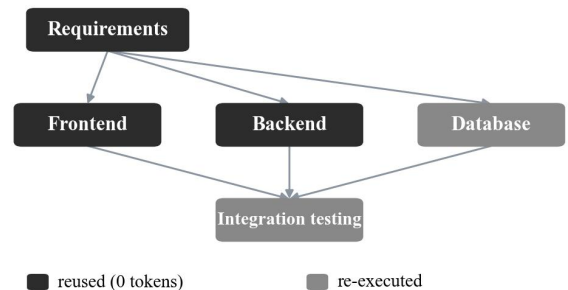


Fig. 3. Selective re-execution after a database requirement change.

E. Capability-Based Tool Routing

When a step actually runs, the Tool Manager resolves its declared capability down to a real provider. If that provider errors out or can't be reached, it just moves to the next one in the chain without marking the step as failed — as long as an alternative exists, the step keeps going. Every chain bottoms out in a local tool that needs no network at all, so a workflow can still finish even if every remote service is down, though with less to show for it. Whenever a substitution happens it gets written to the run log, so the degraded path is there to look at afterwards, not silently swallowed.

F. Human-in-the-Loop Gate

Tools flag whether a call they're about to make can be undone — sending an email, pushing a commit, running a migration, that sort of thing. When one of these comes up, execution stops and shows the exact payload it's about to send, and waits. The payload stays exactly as it was while it waits, so approving it just resumes execution rather than asking the model to generate it again. That makes approval free in token terms, which matters — it takes away the reason someone would skip the checkpoint in a system where confirming something means paying to regenerate it.

G. Orchestration Algorithm

```

Input : task T, prior run state R (may be empty)
Output: completed workflow W
1: G <- Plan(T)           // validated DAG
2: A <- SynthesiseAgents(G)
3: If not RequirementsMet(G): report; halt
4: For each level L in TopoLevels(G) do
5:   For each step s in L in parallel do
6:     sig <- Hash(D(s), Outputs(dep(s)))
7:     If forced(s) or R[s]=NULL or sig != R[s].sig:
8:       t <- SelectTool(capability(s))
9:       If irreversible(t,s): AwaitApproval(s)
10:      O <- Execute(s, t, A[role(s)])
11:      If failed: t2 <- Fallback(capability(s));
12:      O <- Execute(s, t2, A[role(s)])
13:      R[s] <- (sig, O)
14:   Else: O <- R[s].O      // reuse, 0 tokens
15: Return W from R

```

H. Complexity

For a graph with n steps and e edges, both topological ordering and level assignment run in $O(n + e)$. Signatures hashes each step's definition once and each dependency output once, so the total hashing work is also $O(n + e)$. All of this is linear in the size of the graph and has nothing to do with how expensive the steps themselves are, which is dominated by model inference anyway. In practice this means the bookkeeping is essentially free: skipping a single step saves one whole model call, which dwarfs whatever it cost to check whether that step needed to run.

V. RESULTS AND DISCUSSION

A. Experimental Setup

We compared the system against three alternatives. A restart-all policy that just re-runs everything on any change. An equivalent workflow built in LangGraph [10]. And an ablation of our own system, where we swap the output signature for downstream-cone invalidation — the conventional approach, where changing a step marks its entire transitive dependency cone dirty regardless of what that step actually produces. All four run identical agent work on a deterministic offline model with a fixed 0.05 s simulated call latency, so any difference we measure comes from the scheduling policy and nothing else. Step counts, call

counts and token counts are exact and reproduce every time; wall-clock numbers are the median of three runs and, being wall-clock, stay machine-dependent.

Table II lists the seven scenarios we used: a cold run, a change at each of three different depths in the graph, a re-run that happens to produce identical output, a tool going down mid-workflow, and a larger eight-step graph to check the pattern holds at scale.

TABLE II: Benchmark Scenario Definitions

Scenario	Description
full_run	Cold start; execute all five steps once
early_change	Requirement changes at the root, invalidating almost everything
mid_change	Client switches database mid-project (the classic case)
leaf_change	Change at a leaf; nothing downstream depends on it
noop_rerun	A mid-graph step re-runs but produces an identical result
tool_failure	The GitHub API becomes unavailable mid-workflow
microservices_change	Eight-step microservices build; one service's requirements change

B. Per-Scenario Results

Fig. 4 shows step executions across all seven. In full_run and early_change all four systems land on the same number: nothing is cached yet on a cold start, and when the root changes, every signature downstream changes with it, so the whole graph really is invalid. Getting no saving in early_change isn't a shortcoming of the method — reusing anything there would just hand back stale results. The systems start to pull apart from mid_change onward, and the gap widens the further the change sits from the root.

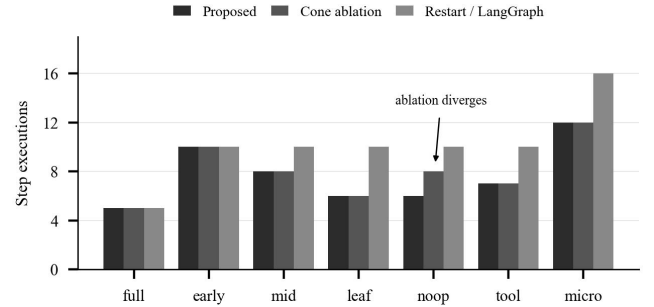


Fig. 4. Step executions per scenario across the four systems.

C. Aggregate Results

TABLE III: Aggregate Results Across Seven Scenarios

Metric	Proposed	Cone	Restart	LangG.
Step executions	54	56	71	71
Tokens consumed	19,196	20,016	24,502	24,502
Wall time (s)	2.76	2.90	4.46	3.37
vs proposed (steps)	—	+4%	+24%	+24%
vs proposed (tokens)	—	+4%	+22%	+22%

Add up all seven scenarios and our scheduler runs 54 steps against 71 for both outside baselines — a 24% cut — and burns 19,196 tokens against 24,502, a 22% cut. Wall time drops 38% against restart-all and 18% against LangGraph, though those wall-clock numbers carry the simulated latency along with them and should be taken as a rough indication rather than a precise figure. Restart-all and LangGraph land on exactly the same numbers because LangGraph has no reuse criterion of its own — it re-runs

the same nodes an explicit restart would. What LangGraph buys you over a plain restart is scheduling efficiency, not less work.

D. Ablation: Signatures versus Dependency Cones

The third system in the comparison is there specifically to test the paper's central claim. It keeps the same DAG, the same parallel level scheduling, the same caching layer — the only thing that changes is the invalidation rule. Instead of comparing output signatures, it just marks the entire transitive cone of a changed step dirty, which is what you'd do without this idea at all.

On six of the seven scenarios, it makes no difference at all. The one place it shows up is `noop_rerun`, where the cone version runs eight steps against our six. That gap is exactly the termination property from Section IV-C at work: when the re-run step reproduces what it produced last time, dependent signatures don't move and propagation just stops, whereas cone invalidation already marked those same dependents dirty based on structure alone and has no way to walk that back. Overall this costs the cone version 4% more, in both steps and tokens.

That 4% looks small here because only one of seven scenarios actually triggers it. How much it matters in practice depends on how often re-execution turns out to be idempotent — which for agent workflows covers retried flaky steps, edits that turn out not to change anything, and re-runs triggered by some unrelated part of the graph changing. What this ablation establishes is the mechanism, not how often it fires: hashing outputs is never worse than hashing structure, and sometimes it's meaningfully better.

E. Sensitivity to Change Position

The 24% aggregate hides a pattern worth spelling out, because it's really what decides whether this method is worth using on a given project. In `early_change`, a root-level change invalidates everything and the saving is zero. In `mid_change` it's 20% of steps. In `leaf_change` and `noop_rerun` it's up to 40%. So the 24% we quote is a midpoint, not a guarantee — where you land in that 0–40% range depends entirely on graph shape and where the change happens to fall. Put another way, this method cuts work it can prove is unnecessary; it doesn't promise a flat speedup. That said, both graph depth and how late changes tend to arrive only grow as projects get bigger, and the largest graph we tested — the eight-step microservices scenario — already saved 25% of steps.

F. Tool-Failure Recovery

In `tool_failure`, the GitHub API drops out partway through the run. Both baselines just record a failed step and stop there, waiting on someone to step in. Our system instead resolves the affected capability to whatever's next in its fallback chain and keeps going, finishing in seven steps instead of ten. The substitution gets logged, so if the run took a degraded path, that's visible afterwards rather than hidden.

G. Implementation and Validation

The prototype covers the planner, dependency graph, agent manager, tool manager, memory, SQLite persistence and event bus, and it's backed by 610+ automated tests at around 90% statement coverage. It talks to live Model Context Protocol servers over stdio, SSE and HTTP, and a Playwright server alone contributes 24 browser tools to the agent pool. On the tool side it actually integrates with GitHub's REST API, PostgreSQL, SMTP mail, and authenticated API connections rather than stubbing any of them out. One end-to-end demo opens a real web page in a real browser, pulls content off it, drafts a message, and sends it over SMTP — pausing for approval right before the send. Underneath, six different model providers sit behind one common interface,

including the deterministic offline one we used for every number reported here.

H. Threats to Validity

We used a deterministic offline model on purpose, to isolate what the scheduling policy alone contributes. With a real, stochastic model, a re-executed step could produce a slightly different output from identical inputs, which would chip away at the termination property from Section IV-C and shrink the margin the ablation measured — that's exactly the gap the semantic-signing idea in Section VI is meant to address. The seven scenarios themselves are synthetic and modelled on software builds, so how much of this carries over to other kinds of tasks is still an open question. The wall-clock numbers bake in a simulated latency and shouldn't be mistaken for real production throughput. And we haven't benchmarked against AutoGen or CrewAI yet, so we're not making any claim about how this compares to either of them.

VI. FUTURE WORK

The most obvious next step is extending the benchmark to AutoGen and CrewAI, since those are the two frameworks people will actually ask about. The harness doesn't care which baseline it's measuring, so this is mostly implementation work rather than anything conceptually new.

A more interesting direction is semantic signing — treating two outputs as the same if they mean the same thing even when the text differs, using embedding similarity above some threshold, say. That would let this whole approach work with a real, stochastic model instead of only the deterministic one we tested with, since exact-match hashing falls apart the moment two runs of the same prompt come back slightly different. It's not free, though — a similarity threshold can be wrong, so this would need a real study of where the threshold sits between saving work and reusing something that's actually gone stale.

Beyond that, we'd like signatures to persist across sessions rather than just within one workflow's re-runs, so related tasks could reuse each other's work and the memory layer starts acting more like a cache than a log. And a cost model that weighs the price of re-executing against the risk of reusing something slightly stale would let the scheduler actually make that trade-off instead of what it does now, which is always re-run when it isn't sure.

VII. CONCLUSION

We set out to build an orchestrator that only re-runs the part of a workflow a change actually invalidates, and that's what this comes down to. Signatures each step over its own definition and its dependencies' exact outputs means invalidation spreads on its own, with no impact rule to write or maintain, and it stops as soon as a re-run gives back the same answer as before. On top of that, the orchestrator routes tools by capability with a fallback chain that always ends somewhere local, and it holds irreversible actions for approval without spending any extra tokens to do it.

Across seven scenarios, against a restart-all baseline and an equivalent LangGraph build, this cut step executions by 24% and tokens by 22%, going as high as 40% when a change lands late in the graph and dropping to zero when a change genuinely invalidates everything — which is the correct thing to happen in that case, not a failure of the method. Swapping the output signature for ordinary dependency-cone invalidation in an ablation cost 4% more in both steps and tokens, which is a small but real confirmation that hashing what a step produced, not just what it's connected to, is what lets the whole thing terminate early. None of

this is theoretical — 610+ tests back the implementation, and it talks to live MCP servers and real external services rather than mocks of them.

If there's a bigger point here, it's that the frameworks we looked at all treat adaptation as something that happens after a failure. In practice, the thing that happens far more often is a requirement changing while nothing has failed at all — and answering that properly means looking at what a step actually produced, not just at where it sits in the graph.

REFERENCES

- [1] L. Deng, J. Yan, C. Cao, M. Zhao and S. Jin, “Dynamic Workflow Orchestration with Executability Verification for Multi-Agent AutoML Task Execution,” in 2026 6th Int. Conf. on Artificial Intelligence, Big Data and Algorithms (CAIBDA), IEEE, 2026, doi: 10.1109/CAIBDA70336.2026.11621590.
- [2] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework,” arXiv:2308.08155, 2023.
- [3] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” arXiv:2308.00352, 2023.
- [4] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022.
- [5] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” arXiv:2303.11366, 2023.
- [6] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” arXiv:2302.04761, 2023.
- [7] Y. Shen et al., “HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face,” arXiv:2303.17580, 2023.
- [8] N. Otoum and N. Elkhaili, “Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review,” IEEE Access, 2026, doi: 10.1109/ACCESS.2026.3652325.
- [9] Anthropic, “Model Context Protocol Specification,” 2024. [Online]. Available: modelcontextprotocol.io
- [10] LangChain, “LangGraph Documentation,” 2024. [Online]. Available: langchain-ai.github.io/langgraph